Selenium Automation Report

Automation Testing of Tichi Web Application

1. Introduction

Testing the Login and Signup pages manually every time a change is made can take a lot of time, and it is easy to miss a step when the same checks are repeated again and again. To make this process faster and more consistent, Selenium Automation was used for this project. Selenium allows the browser to be controlled through a Python script, so the same set of steps can be run automatically instead of doing them by hand each time.

2. Objective

The main objective of this project was to automate the Login and Signup functionality of the Tichi web application so that these two modules could be tested quickly and reliably without manual effort. Along with this, the goal was to build a simple data-driven setup where test data comes from an Excel file, so that new test cases can be added later just by adding a new row, without changing the test script itself.

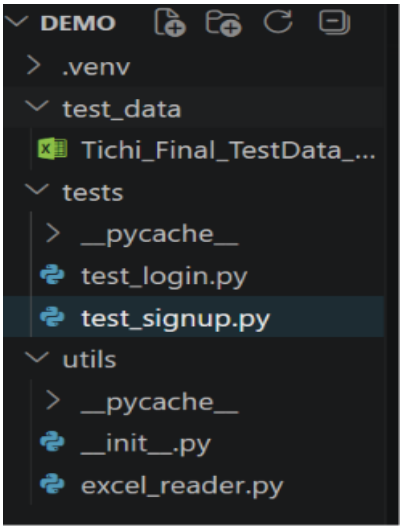
3. Automation Scope

- Login Module — validating login behaviour for different combinations of email and password provided through Excel.
- Signup Module — validating the registration form, including filling details, accepting terms, and creating an account.

4. Tools Used

Tool	Purpose
Python	Programming Language
Selenium	Automation Tool
openpyxl	Excel Handling
webdriver-manager	Chrome Driver Management
VS Code	Development IDE
Chrome	Browser

5. Project Structure



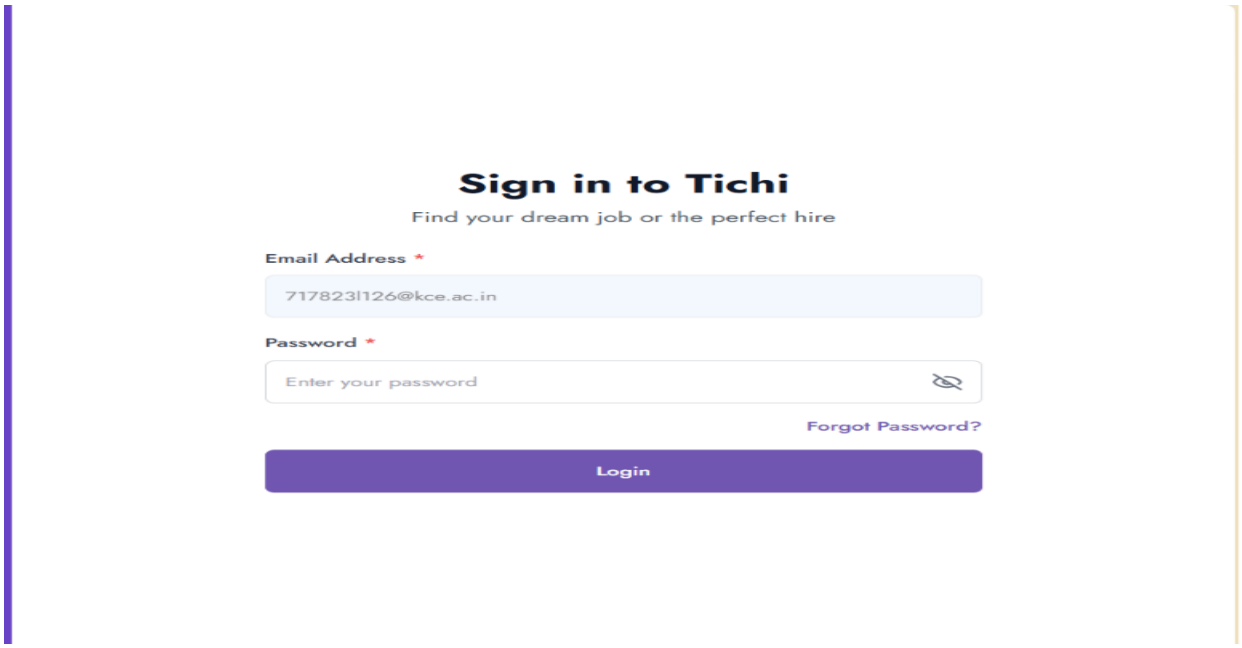
6. Automation Workflow

The overall flow followed by the framework for both Login and Signup is shown below:

Read Excel
↓
Read Login / Signup Test Data
↓
Launch Chrome Browser
↓
Execute Selenium Script
↓
Capture Actual Result
↓
Compare Expected vs Actual
↓
Write PASS / FAIL into Excel
↓
Store Execution Time

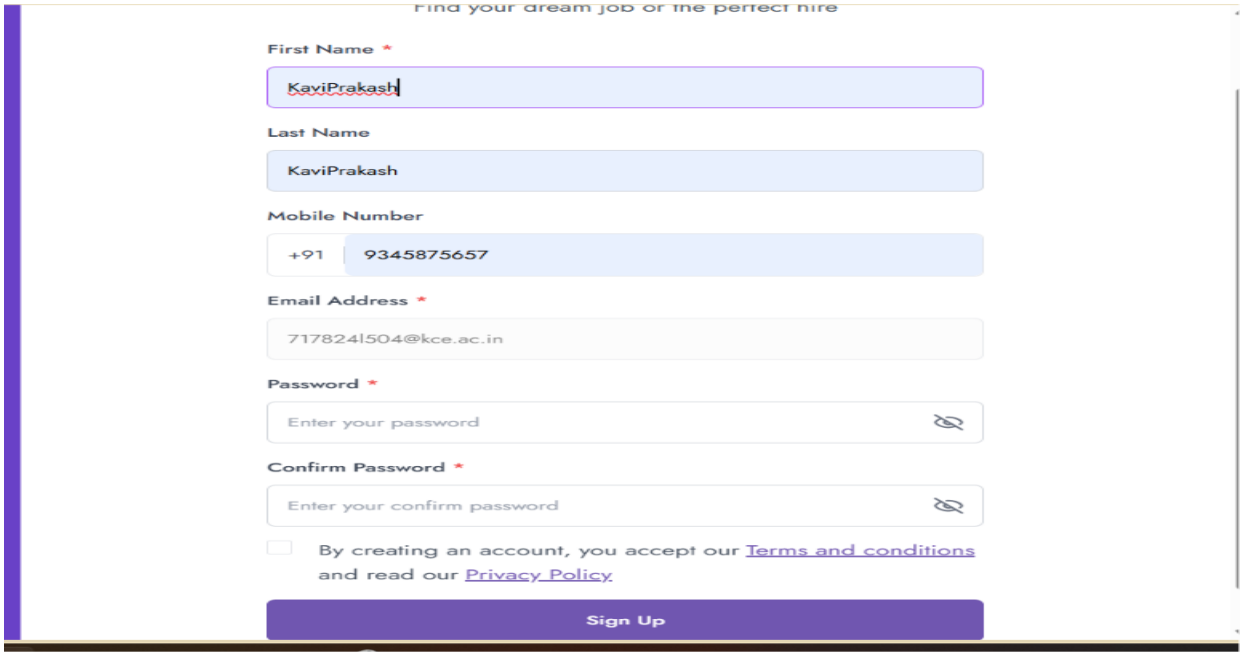
7. Login Automation

For the Login module, the script reads the email and password for each row from the Excel sheet using openpyxl. Selenium then opens the Tichi login page and enters the email first, clicks Continue, and then enters the password on the next step before clicking Login. Once the login attempt is complete, the script captures the actual result (for example, whether the dashboard loaded or an error message appeared) and compares it with the expected result mentioned in the Excel sheet. Based on this comparison, the PASS or FAIL status is written back into the same row in Excel.



8. Signup Automation

For the Signup module, the script reads the registration details (name, email, password, etc.) from Excel and fills the registration form on the Tichi signup page using Selenium. It also checks the Accept Terms checkbox, if present, before clicking the Create Account button. After submitting the form, the actual result is compared with the expected result from Excel, and the PASS or FAIL status along with the result is updated back into the Excel sheet, the same way as it is done for Login.



9. Data-Driven Testing

Instead of writing separate test methods for every single combination of Login or Signup data, the framework reads all test cases directly from an Excel file. For every row read from Excel, the script runs the same Selenium steps but with different data. After execution, three things are written back into the same Excel file for each row: the Actual Result observed during execution, the PASS or FAIL status based on comparison with the Expected Result, and the Execution Time taken to run that particular test case. This keeps the test data and the test logic separate, which makes it much easier to add more test cases later.

Screenshot 3

Excel Before Execution

TC_ID	Scenario	Email	Password	Expected Result	Actual Result	Status	Execution Time (sec)
TC_LOGIN_001	Valid Login	717823l126@kce.ac.in	Kaviprakash@12	Login Successful			
TC_LOGIN_002	Invalid Password	717823l126@kce.ac.in	WrongPass@123	Error Message			
TC_LOGIN_003	Invalid Email	wronguser@gmail.com	Kaviprakash@12	Error Message			
TC_LOGIN_004	Invalid Email Format	abcgmail.com	Kaviprakash@12	Email Validation Message			
TC_LOGIN_005	Empty Email		Kaviprakash@12	Email Required			
TC_LOGIN_006	Empty Password	717823l126@kce.ac.in		Password Required			
TC_LOGIN_007	Both Fields Empty			Validation Messages			

Screenshot 4

Excel After Execution

TC_ID	Scenario	Email	Password	Confirm Password	Expected Result	Actual Result	Status	Execution Time (sec)
TC_SIGNUP_001	Valid Signup	newuser01@test.com	Password@123	Password@123	Account Created	Message: Stacktrace:chromedriver!GetHandle	PASS	20.69
TC_SIGNUP_002	Invalid Email	abcgmail.com	Password@123	Password@123	Email Validation Message	Message: Stacktrace:chromedriver!GetHandle	PASS	16.51
TC_SIGNUP_003	Password Mismatch	newuser02@test.com	Password@123	Password@124	Password Mismatch Message	Message: element click intercepted: Element	PASS	34.92
TC_SIGNUP_004	Empty Email		Password@123	Password@123	Email Required	Message: Stacktrace:chromedriver!GetHandle	PASS	15.01
TC_SIGNUP_005	Empty Password	newuser03@test.com			Password Required	Message: element click intercepted: Element	PASS	11.35
TC_SIGNUP_006	Existing Email	717823l126@kce.ac.in	Password@123	Password@123	Account Already Exists	Message: Stacktrace:chromedriver!GetHandle	PASS	15.67
TC_SIGNUP_007	All Fields Empty				Required Field Validation	Message: Stacktrace:chromedriver!GetHandle	PASS	14.28

10. Challenges Faced

- Identifying stable XPath locators for the Login and Signup fields, since some elements did not have a fixed ID.
- Handling Explicit Waits correctly so that Selenium waits for the right element instead of failing on slower page loads.
- Reading and updating the same Excel file using openpyxl without corrupting the existing data or formatting.
- Managing dynamic web elements that took a moment to appear after clicking Continue or Create Account.

- Validating Expected vs Actual Results correctly, especially when the actual message on the page was worded slightly differently than expected.

11. Conclusion

Overall, the Login and Signup automation for the Tichi web application was implemented successfully using Selenium WebDriver along with a simple data-driven approach based on Excel. This setup made it possible to run multiple test cases for both modules without repeating manual steps, and to keep a clear record of results, including PASS/FAIL status and execution time, directly inside the Excel file. This project helped in understanding how Selenium, Python, and Excel can be combined to build a small but practical automation framework.